

CMS Hospital Price Transparency — Research & Build Plan

Overview

Before writing any application code, this plan validates the CMS data landscape through structured research phases. Each phase has a clear success gate before proceeding. The build expands in scope only as data quality is confirmed.

Context: What We Know Going In

- Since January 2021, all U.S. hospitals must publish machine-readable files (MRFs) with standard charges
 - As of 2026, CMS enforces a **standardized template** (CSV tall, CSV wide, or JSON) — this is a major improvement over early years when every hospital rolled their own format
 - Required fields in every compliant MRF: gross charge, discounted cash price, payer-specific negotiated charge, de-identified min/max negotiated charge
 - CMS enforcement tightened April 1, 2026 (CY2026 OPPS/ASC Final Rule) — compliance and data quality should be improving
 - Each hospital publishes its own MRF on its website; there is **no single central download** — hospitals post a `cms-hpt.txt` file at their domain root that points to the MRF location
 - The official CMS GitHub (`CMSgov/hospital-price-transparency`) has the template specs and a validator tool
-

Phase 1 — Schema & Format Research

Goal: Understand exactly what's in a compliant MRF before touching any data pipeline code.

1A — Read the official spec

- ☐ Read the CMS data dictionary at `github.com/CMSgov/hospital-price-transparency`
- ☐ Note all required vs optional fields in the 2026 template
- ☐ Understand the difference between CSV tall, CSV wide, and JSON formats
- ☐ Document which fields are relevant to us: `description`, `code_type`, `code`, `gross_charge`, `cash_discounted_price`, `payer_name`, `plan_name`, `standard_charge_negotiated_dollar`

1B — Locate MRFs via cms-hpt.txt discovery

Each hospital publishes a txt file at `https://[hospital-domain]/cms-hpt.txt` that links to their MRF. Test this discovery method on 3 hospitals before assuming it works reliably.

- ☐ Pick a large well-known hospital (e.g. Mayo Clinic, Cleveland Clinic, Johns Hopkins)
- ☐ Navigate to `[domain]/cms-hpt.txt` and confirm it resolves and points to an MRF
- ☐ Download the MRF and inspect: what format is it (CSV/JSON)? How large is the file?
- ☐ Note any quirks: encoding issues, nested structures, empty fields

Validation gate: If `cms-hpt.txt` discovery works on 3/3 hospitals → proceed. If it fails on 2+, research alternative discovery methods before continuing.

Phase 2 — Sample City Research (5 Cities)

Run this research across five geographically diverse cities. The goal is to understand data quality and format consistency across different hospital systems and regions.

Target cities

City	Why chosen
New York, NY	Largest market, high price variation expected
Houston, TX	Large uninsured population, different market dynamics
Minneapolis, MN	Strong integrated health systems (Allina, Fairview)
Phoenix, AZ	Fast-growing market, mix of large and regional hospitals
Asheville, NC	Mid-size regional market, tests non-major-metro coverage

Per-city research tasks

For each city, identify 3–5 hospitals and for each:

- ☐ Confirm `cms-hpt.txt` resolves and MRF is accessible
- ☐ Document file format (CSV tall / CSV wide / JSON)
- ☐ Document file size (small files may mean incomplete data)
- ☐ Pick 3 test procedures and search for them:
 - Hip replacement (DRG 470)
 - Vaginal delivery (DRG 807)

- Colonoscopy (CPT 45378)
- ☐ Record: is the procedure found? What price fields are populated? Are negotiated rates present or blank?
- ☐ Note data quality issues: placeholder values (old "999999999"), missing descriptions, inconsistent code formats

City research output

For each city, produce a short summary:

- Number of hospitals researched
- % with accessible MRFs
- % with populated cash price field
- % with at least one negotiated rate present
- Data format breakdown (CSV vs JSON)
- Any city-specific anomalies

Validation gate: If 3+ of 5 cities show >70% of hospitals with accessible, populated MRFs → the direct-fetch approach is viable. If not → evaluate aggregator options (see Phase 4).

Phase 3 — Code & Billing Mapping Research

Goal: Confirm how to map plain-language procedure names to the right codes in MRF data.

3A — Understand code types in MRFs

MRFs use multiple code types. Document which appear most frequently in the sampled files:

- ☐ DRG (inpatient bundles — most common for hospital stays)
- ☐ CPT / HCPCS (outpatient procedures)
- ☐ APC (Ambulatory Payment Classifications)
- ☐ RC (Revenue Codes)
- ☐ NDC (drug codes — less relevant for procedures)

Key question: For our target use case (patients comparing procedure costs), are DRG codes sufficient for inpatient, or do we need CPT too?

3B — Build a starter mapping table

Create a CSV with 30 common hospital procedures covering both inpatient and outpatient:

Columns: (plain_language_name | primary_code_type | primary_code | alternate_codes | notes)

Example rows:

Hip replacement	DRG	470	CPT 27130	Inpatient; DRG 470 = without MCC
Knee replacement	DRG	470	CPT 27447	Same DRG as hip for billing
Colonoscopy	CPT	45378	HCPCS G0121	Outpatient; G0121 = screening
C-section	DRG	783	CPT 59510	DRG varies by complications
Appendectomy	DRG	341	CPT 44950	DRG 341 = laparoscopic
Cardiac catheterization	DRG	287	CPT 93454	Multiple DRGs by complexity

- ☐ Research and complete the 30-procedure mapping table
- ☐ For each procedure, test whether searching by code returns results in the sampled MRFs
- ☐ Note any procedures where MRF data is consistently absent

3C — Evaluate Claude API as fallback

- ☐ Draft a prompt for mapping natural language → CPT/DRG codes
- ☐ Test with 10 ambiguous inputs ("stomach surgery", "heart stent", "brain scan")
- ☐ Evaluate response quality: does it return the right codes? Does it handle ambiguity well?
- ☐ Document prompt template that works reliably

Validation gate: If the 30-procedure table covers >80% of expected user queries AND Claude fallback handles the rest reliably → translation layer is viable without a third-party medical terminology API.

Phase 4 — Aggregator Evaluation (Conditional)

Run this phase only if Phase 2 shows direct MRF fetching is too unreliable, OR if you want to evaluate cost vs build effort tradeoffs.

Options to evaluate

Aggregator	Model	What it solves
Turquoise Health	Paid API (~enterprise pricing)	Pre-normalized data, negotiated rates, updated monthly
Ribbon Health	Paid API	Provider directory + pricing combined
RAND Hospital Price Transparency	Free research dataset	Annual snapshots, good for benchmarking not real-time

Aggregator	Model	What it solves
CMS Care Compare API	Free	Quality ratings only, no pricing
data.cms.gov datasets	Free	Enforcement data, some pricing aggregates

Research tasks

- ☐ Request pricing/trial access from Turquoise Health and Ribbon Health
- ☐ Download the latest RAND dataset and assess freshness and coverage
- ☐ Check data.cms.gov for any centralized MRF index or aggregated pricing dataset
- ☐ Evaluate: does the cost of a paid aggregator justify the saved engineering time for MVP?

Decision point: If a free or low-cost aggregator covers your 5 target cities well → use it for MVP and build direct MRF fetching as a later optimization. If aggregators are enterprise-only pricing → build the direct pipeline.

Phase 5 — Pipeline Architecture Decision

After phases 1-4, make a documented decision on data architecture before any code:

Option A — Direct MRF fetching (per hospital)

- Discover hospital MRF URLs via `cms-hpt.txt`
- Download and parse MRF on demand (or on a scheduled basis)
- Store normalized results in your own database
- Best if: MRF quality is good and you want no external dependencies

Option B — Aggregator-first

- Query Turquoise Health or similar via API
- Use their normalized, deduplicated data
- Best if: engineering speed matters more than cost at MVP stage

Option C — Hybrid

- Use aggregator for major metro hospitals (best data quality)
- Fall back to direct MRF fetch for smaller/regional hospitals
- Best for: maximizing coverage at launch

Document the chosen approach and rationale before starting Phase 6.

Phase 6 — MVP Build (Scope-Gated)

Build in layers. Expand scope only when the previous layer is validated.

Layer 1 — Single city, single procedure (proof of concept)

- ☐ Pick New York as first city (most hospitals, highest stakes for data quality)
- ☐ Pick hip replacement (DRG 470) as first procedure (common, well-documented)
- ☐ Build: input → code lookup → MRF fetch → parse → display results for NY hospitals
- ☐ Validate: are results returning real, plausible prices? Are there obvious data errors?
- ☐ Show results to 2–3 real people and collect feedback on clarity

Gate: 5+ NY hospitals returning plausible hip replacement prices → expand to Layer 2.

Layer 2 — Single city, 10 procedures

- ☐ Expand procedure mapping to 10 procedures covering inpatient and outpatient
- ☐ Validate each new procedure returns results for NY hospitals
- ☐ Handle edge cases: procedure not found, price field empty, multiple code matches
- ☐ Add basic UI: search input, results list with hospital name, address, cash price, distance

Gate: 8/10 procedures returning results with >3 hospitals each → expand to Layer 3.

Layer 3 — 5 cities, 10 procedures

- ☐ Roll out to all 5 research cities
- ☐ Validate data quality per city matches Phase 2 findings
- ☐ Add location detection / ZIP code input
- ☐ Add sorting: by price, by distance, by quality rating (CMS Care Compare)
- ☐ Surface data quality indicator per result ("price reported as of [date]")

Gate: All 5 cities returning consistent results, no major data gaps → expand to Layer 4.

Layer 4 — National coverage, 30 procedures

- ☐ Expand to all discoverable hospitals nationally via `cms-hpt.txt` crawl
 - ☐ Expand procedure list to 30
 - ☐ Build scheduled refresh pipeline (MRFs update at least annually; many update more often)
 - ☐ Add disclaimer UI (cash price vs insured price explanation)
 - ☐ Prepare for public launch
-

Key URLs

- CMS Hospital Price Transparency overview: cms.gov/priorities/key-initiatives/hospital-price-transparency
- CMS template spec and data dictionary: github.com/CMSgov/hospital-price-transparency
- CMS MRF validator tool: cmsgov.github.io/hpt-tool/
- CMS Care Compare API (quality ratings): data.cms.gov/provider-data
- NPI Registry (provider lookup): npiregistry.cms.hhs.gov/api
- CMS enforcement data: data.cms.gov/provider-characteristics/hospitals-and-other-facilities/hospital-price-transparency-enforcement-activities-and-outcomes

Required disclaimers for public site

1. Prices shown are the hospital's published standard charges, not what a specific insured patient will pay
2. Actual out-of-pocket cost depends on insurance plan, deductible, and in-network status
3. Data is sourced from hospital-published machine-readable files and may not reflect the most current pricing
4. This tool is for informational purposes only and does not constitute medical or financial advice

Notes on 2026 CMS updates

- New CY2026 requirements (enforced April 1, 2026) added and removed several MRF data elements
- If parsing MRFs, validate against the 2026 template spec, not earlier versions
- Hospitals that haven't updated to the 2026 schema may have schema mismatches — your parser needs to handle both